GUIA DO ALUNO

Sensores: medindo o mundo com Arduino e ESP32

DS18B20 · DHT11 · MQ-2 · MQ-135 · BME680 · pH-4502C — física, química, calibração, bibliotecas e monitor Wi-Fi no celular

---

Placas: Arduino Uno → ESP32 • **6 sensores, 6 aulas** • cada aula termina com o sensor no celular pelo Wi-Fi

Pré-requisitos: aulas de LEDs, buzzer e sensor de distância (LED RGB, `millis()`, serial, página web)

Carga horária: 6 aulas de 100 min

Espaço Cria — 2026

Sumário

Como este curso funciona

Segurança e cuidados gerais

Aula 1 — DS18B20: termômetro digital à prova d'água

Aula 2 — DHT11: temperatura e umidade do ar

Aula 3 — MQ-2: gás inflamável e fumaça

Aula 4 — MQ-135: qualidade do ar

Aula 5 — BME680: estação meteorológica num chip

Aula 6 — pH-4502C: medindo a acidez

Comparando os seis sensores

Glossário

Anexo — Ficha de calibração

Anexo — Checklist

Como este curso funciona

Um **sensor** transforma uma grandeza física ou química (temperatura, umidade, gás, pressão, acidez) num sinal elétrico que o microcontrolador consegue ler. Em cada aula vamos responder às mesmas perguntas para um sensor diferente: **o que ele mede e como** (a física/química por trás), **como ligar**, **como ler com a biblioteca padrão**, **como calibrar**, **para que serve na vida real** — e terminar com o sensor mandando dados para o **celular pelo Wi-Fi** do ESP32.

Aula	Sensor	Mede	Princípio	Interface	Biblioteca
1	DS18B20	Temperatura (sonda à prova d'água)	Junção de silício + ADC interno	1-Wire (1 fio de dados)	OneWire + DallasTemperature
2	DHT11	Temperatura e umidade do ar	Polímero capacitivo + termistor NTC	1 fio, protocolo próprio	DHT sensor library
3	MQ-2	Gás inflamável (GLP, H_2, CO) e fumaça	SnO_2 aquecido: resistência cai com o gás	Analógica (AO) + digital (DO)	MQUnifiedsensor
4	MQ-135	Qualidade do ar (CO_2 estimado, NH_3, álcool)	Idem, dopado para outros gases	Analógica + digital	MQUnifiedsensor
5	BME680	Temperatura, umidade, pressão, VOC	4 sensores MEMS num chip	I^2C (2 fios)	Adafruit BME680
6	pH-4502C	Acidez de líquidos (pH 0–14)	Eletrodo de vidro — equação de Nernst	Analógica (BNC → placa)	nenhuma: nós calibramos

Como usar este guia

Cada aula: **o que é e para que serve → a física/química → montagem (Uno) → código → calibração → ESP32 + celular → experimente → desafios**. Os desafios sempre incluem um **sistema de alerta com LED RGB e buzzer** — você já sabe fazer isso das aulas anteriores; agora o gatilho é o sensor.

Códigos em `aula-sensores/codigo/`: `NN_sensor_uno` e `NNb_sensor_esp32_web`. Os sketches do ESP32 precisam dos arquivos `monitor_wifi.h` e `pagina_web.h` **na mesma pasta** (a IDE os abre como abas).

Segurança e cuidados gerais

- **3,3 V no ESP32:** sensores alimentados em 5 V que devolvem sinal de 5 V (MQ-2, MQ-135, pH-4502C) precisam do **divisor de tensão 1 kΩ/2 kΩ** antes do GPIO. Sem isso o pino queima.
- Os sensores MQ têm um **aquecedor** que esquenta (~50 °C na carcaça) e puxa ~150 mA: alimente pelo 5 V/VIN, nunca pelo 3V3.
- A sonda de pH é de **vidro**: não bata, não deixe secar (guarde na solução de armazenamento ou KCl 3 M), enxágue com água destilada entre soluções.
- Soluções tampão e amostras: use luvas se testar produtos de limpeza; nunca prove nada; descarte na pia com água corrente.
- Gás para testar o MQ-2: **isqueiro sem acender** (só o gás) ou álcool num algodão a 10 cm — nunca chama perto do sensor.
- Desligue o USB antes de mexer nos fios; confira polaridade duas vezes (VCC/GND invertido é a morte nº 1 dos sensores).

Aula 1 — DS18B20: termômetro digital à prova d'água

O que é e para que serve

O **DS18B20** é um termômetro digital da Maxim/Dallas encapsulado num tubo de aço inox com cabo de 1 m: mede de **−55 °C a +125 °C** com precisão de **±0,5 °C** (entre −10 e 85 °C). Serve para aquários, caixas d'água, compostagem, chocadeira, solo, geladeira, fermentação de cerveja — qualquer lugar onde um sensor de ar não pode ficar. Cada chip tem um **número de série de 64 bits** gravado de fábrica: dá para ligar dezenas de sensores no **mesmo fio** e saber quem é quem.

A física: como um chip "sente" temperatura

Dentro do tubo há um chip de silício. Uma junção de silício (como a de um diodo) tem uma tensão que **cai ~2 mV para cada grau** que a temperatura sobe — é um efeito físico previsível e linear. O chip compara essa tensão com uma referência interna estável, converte com um **ADC de 9 a 12 bits** (12 bits = passos de 0,0625 °C) e entrega o número pronto. Por isso ele é **digital**: quem converte é o próprio sensor, não o Arduino; o número não sofre com ruído nem com o comprimento do cabo.

1-Wire: o protocolo usa um único fio de dados (DQ) com resistor de **pull-up de 4,7 kΩ** para 5 V. O Arduino "puxa" o fio para 0 V em pulsos curtos para falar; o sensor responde do mesmo jeito. O resistor é o que devolve o fio para nível alto quando ninguém está falando — **sem ele, nada funciona** (leitura −127 °C).

Montagem

Fio da sonda	Arduino Uno	ESP32
Vermelho (VDD)	5V	3V3
Preto (GND)	GND	GND
Amarelo (DQ)	pino 2	GPIO 4
Resistor 4,7 kΩ	entre DQ e 5V	entre DQ e 3V3

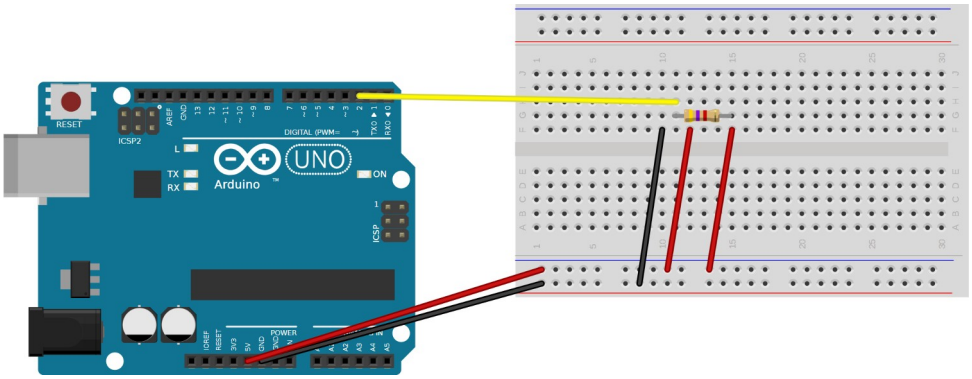


Figura 1 — Aula 1 — DS18B20 no Uno: sonda na protoboard, pull-up de 4,7 kΩ entre DQ e +5 V, DQ no pino 2.

Código (Uno)

01_ds18b20_uno.ino

```
/*
  01 - DS18B20: termometro digital a prova d'agua (1-Wire)
  Placa: Arduino Uno/Nano     Bibliotecas: OneWire, DallasTemperature (Gerenciador de Bibliotecas)

  Ligacoes da sonda (cabo de 3 fios):
```

```
    VERMELHO (VDD) -> 5V      PRETO (GND) -> GND      AMARELO (DQ, dados) -> pino 2
    Resistor de 4,7 kOhm entre DQ e 5V (pull-up). SEM ele o sensor nao responde (-127 C).

  Fisica: dentro do tubo de inox ha um chip que mede a temperatura pela tensao de uma juncao
  de silicio (varia ~2 mV por grau) e a converte com um ADC interno de ate 12 bits (0,0625 C).
  1-Wire: um unico fio serve para alimentar e conversar; cada sensor tem um ENDERECO de 64 bits
  gravado de fabrica, entao varios podem dividir o mesmo fio.
*/
#include <OneWire.h>
#include <DallasTemperature.h>

const int PINO_DADOS = 2;
OneWire oneWire(PINO_DADOS);
DallasTemperature sensores(&oneWire);
DeviceAddress endereco;

void setup() {
  Serial.begin(9600);
  sensores.begin();
  int n = sensores.getDeviceCount();
  Serial.print("Sensores encontrados: "); Serial.println(n);
  for (int i = 0; i < n; i++) {
    if (sensores.getAddress(endereco, i)) {
      Serial.print("  #"); Serial.print(i); Serial.print(" endereco: ");
      for (int b = 0; b < 8; b++) { if (endereco[b] < 16) Serial.print("0"); Serial.print(endereco[b],
HEX); }
      Serial.println();
      sensores.setResolution(endereco, 12);      // 9..12 bits: 12 = 0,0625 C, leva 750 ms
    }
  }
}

void loop() {
  sensores.requestTemperatures();               // manda TODOS os sensores converterem
  for (int i = 0; i < sensores.getDeviceCount(); i++) {
    float t = sensores.getTempCByIndex(i);
    Serial.print("sensor "); Serial.print(i); Serial.print(": ");
    if (t == DEVICE_DISCONNECTED_C) Serial.println("desconectado (confira o pull-up de 4,7k)");
    else { Serial.print(t, 2); Serial.println(" C"); }
  }
  delay(1000);
}
```

Entenda o código

- `OneWire oneWire(2)` cria o “fio”; `DallasTemperature sensores(&oneWire)` sabe conversar com DS18B20 nesse fio.
- `getDeviceCount()` e `getAddress()` mostram **quantos** sensores há e o **endereço** de cada um — ligue dois e veja.
- `setResolution(12)`: mais bits = mais casas decimais, porém a conversão leva 750 ms. Com 9 bits leva 94 ms (0,5 °C).
- `requestTemperatures()` manda todos converterem **ao mesmo tempo**; depois `getTempCByIndex(i)` lê cada um.
- `DEVICE_DISCONNECTED_C` (−127) é o código de erro — quase sempre falta o pull-up ou o fio de dados está solto.

Calibração e verificação

1. **Ponto de gelo:** copo com gelo picado + um pouco de água, mexa 1 min, ponta da sonda dentro: deve marcar **0,0 ± 0,5 °C**.

2. **Ponto de ebulição:** água fervendo ao nível do mar = 100 °C (em Recife, ~99,8 °C; a 800 m de altitude, ~97 °C). Cuidado com a mão.
3. Se houver um desvio constante (ex.: sempre +0,4 °C), some uma correção no código: `t = t - 0.4;` — é uma **calibração de offset**. O DS18B20 raramente precisa.

Monitor Wi-Fi no celular (ESP32)

O sketch do ESP32 cria a rede **Sensor-DS18B20** (senha `12345678`). Conecte o celular a ela (desligue os dados móveis) e abra **http://192.168.4.1**: a página mostra o valor principal em destaque, os demais valores em cartões, um gráfico das últimas 120 leituras e um aviso de **ALERTA** quando o limite do código é ultrapassado.

- A página é a mesma para todos os sensores (`pagina_web.h`): ela pede `/dados` a cada 2 s e o ESP32 responde um **JSON** com uma lista de itens `{n, v, u}` (nome, valor, unidade).
- O servidor está em `monitor_wifi.h`: `monitorIniciar(nome)`, `monitorAtender()` no `loop()` e `monitorPublicar(json)` quando há leitura nova. Para um sensor novo, só muda o que entra no JSON.
- O sketch guarda **máxima e mínima** desde que ligou e usa `setWaitForConversion(false)` para não travar o `loop()` durante os 750 ms da conversão — a mesma ideia do `millis()`.

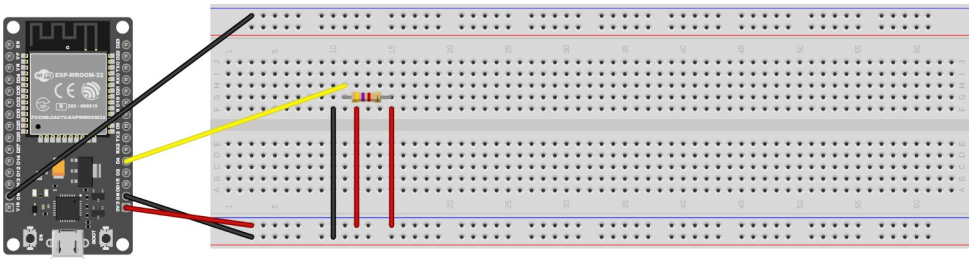


Figura 2 — Aula 1 — DS18B20 no ESP32 (3V3, GPIO 4, pull-up para 3V3).

01b_ds18b20_esp32_web.ino

```
/*
  01b - DS18B20 no ESP32 com monitor Wi-Fi no celular
  Placa: ESP32 Dev Module     Bibliotecas: OneWire, DallasTemperature
  Arquivos extras na pasta do sketch: monitor_wifi.h, pagina_web.h

  Ligacoes: VERMELHO -> 3V3 (o DS18B20 aceita 3,0-5,5 V)   PRETO -> GND   AMARELO -> GPIO 4
            resistor 4,7 kOhm entre GPIO 4 e 3V3
  O que muda do Uno: so o pino e a alimentacao em 3,3 V. A biblioteca e a mesma.
  Celular: rede "Sensor-DS18B20" (senha 12345678) -> http://192.168.4.1
*/
#include <OneWire.h>
#include <DallasTemperature.h>
#include "monitor_wifi.h"

const int PINO_DADOS = 4;
const float LIMITE_ALERTA = 30.0;      // C
OneWire oneWire(PINO_DADOS);
DallasTemperature sensores(&oneWire);
unsigned long ultimaLeitura = 0;
float tMax = -100, tMin = 200;

void setup() {
  Serial.begin(115200);
  sensores.begin();
  sensores.setResolution(12);
  sensores.setWaitForConversion(false);   // nao trava o loop (o Wi-Fi precisa dele livre)
  sensores.requestTemperatures();
  monitorIniciar("Sensor-DS18B20");
}
```

```
void loop() {
  monitorAtender();
  if (millis() - ultimaLeitura >= 1000) {
    ultimaLeitura = millis();
    float t = sensores.getTempCByIndex(0);      // resultado da conversao pedida ha 1 s
    sensores.requestTemperatures();             // pede a proxima
    if (t == DEVICE_DISCONNECTED_C) { monitorPublicar(jsonMonitor("DS18B20", item("Temperatura", 0, "C"),
true, "sensor desconectado")); return; }
    if (t > tMax) tMax = t;
    if (t < tMin) tMin = t;
    bool alerta = t > LIMITE_ALERTA;
    Serial.printf("%.2f C\n", t);
    monitorPublicar(jsonMonitor("DS18B20 - temperatura",
      item("Temperatura", t, "C") + "," + item("Maxima", tMax, "C") + "," + item("Minima", tMin, "C") +
"," + item("Limite", LIMITE_ALERTA, "C"),
      alerta, "temperatura acima de " + String(LIMITE_ALERTA, 0) + " C"));
  }
}
```

Experimente

1. Segure a ponta da sonda: quanto tempo leva para chegar à temperatura da mão? Por que é lento? (massa do tubo de inox)
2. Mude a resolução para 9 bits e observe os “degraus” de 0,5 °C no gráfico do celular.
3. Ligue **dois** sensores no mesmo fio e imprima os dois endereços. Coloque um no gelo e o outro na mão.

Desafios (com alerta por LED RGB e buzzer)

1. **1.1 Termostato de aquário** — abaixo de 24 °C: LED azul; 24–28 °C: verde; acima de 28 °C: vermelho + buzzer intermitente. Use **histerese** de 0,5 °C para não ficar oscilando.
2. **1.2 Alarme de geladeira aberta** — se a temperatura subir mais de 2 °C em 1 min, dispare o buzzer (guarde a leitura de 60 s atrás num vetor circular).
3. **1.3 Dois sensores, um alerta** — sonda de água e sonda de ar: acenda o LED se a diferença passar de 5 °C (choque térmico).
4. **1.4 Registro de máx/mín com botão** — botão zera a máxima e a mínima; buzzer dá um bipe curto ao registrar nova máxima.
5. **1.5 Alerta na página** — mude `LIMITE_ALERTA` por uma rota `/limite?c=27` (como no monitor de distância) e mostre o LED vermelho quando a página marcar ALERTA.

Aula 2 — DHT11: temperatura e umidade do ar

O que é e para que serve

O **DHT11** (azul) mede **umidade relativa de 20 a 90 % (±5 %)** e **temperatura de 0 a 50 °C (±2 °C)**, uma leitura a cada 2 s. É barato e serve para estufas, viveiros, monitorar mofo em salas, incubadoras e estações meteorológicas escolares. O irmão **DHT22** (branco) é mais preciso (±2 % / ±0,5 °C) e usa o mesmo código, trocando `DHT11` por `DHT22`.

A física: capacitância e termistor

Umidade: entre duas placas metálicas há um **polímero** que absorve moléculas de água do ar. Quanto mais água, maior a **capacitância** entre as placas (a água tem constante dielétrica alta, ~80, contra ~3 do polímero seco). Um circuito mede essa capacitância e a converte em % de umidade relativa — a razão entre o vapor presente e o máximo que o ar aguenta naquela temperatura. **Temperatura:** um **termistor NTC**, um resistor de cerâmica cuja resistência **cai** com o calor (Negative Temperature Coefficient). Um chip de 8 bits lê os dois, corrige com a calibração de fábrica e envia **40 bits** (umidade, temperatura, verificação) pelo único fio de dados, num protocolo próprio de pulsos — a biblioteca cuida disso.

As três versões do DHT11 — leia a serigrafia!

Versão	Pinos (esq. → dir.)	Como ligar
Sensor “pelado”, 4 pinos	VCC · DATA · (NC) · GND	resistor 10 kΩ entre DATA e VCC obrigatório; NC fica solto
Módulo 3 pinos “VCC GND OUT” (ou “+ − OUT”)	conforme escrito na placa	resistor já vem na placa; OUT → pino de dados
Módulo 3 pinos “S − (meio)” (KY-015 e similares)	S · (sem nome) · −	S = dados, **meio = VCC**, − = GND

Ordem dos pinos muda de fabricante para fabricante

Alguns módulos vêm “S + −”, outros “− + S”, outros “VCC GND OUT”. **Não decore a posição: leia a placa.** VCC e GND invertidos por alguns segundos costumam matar o DHT11. Na dúvida, o pino do meio quase sempre é VCC nos módulos de 3 pinos.

Montagem

DHT11	Arduino Uno	ESP32
VCC / + / meio	5V	3V3
GND / −	GND	GND
DATA / OUT / S	pino 2	GPIO 4
Pull-up 10 kΩ (só no de 4 pinos)	DATA ↔ 5V	DATA ↔ 3V3

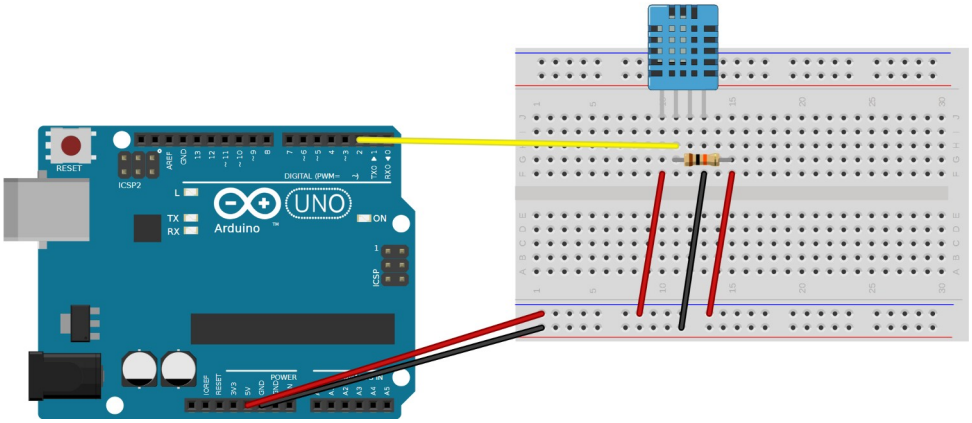


Figura 3 — Aula 2 — DHT11 de 4 pinos no Uno com pull-up de 10 kΩ (nos módulos de 3 pinos o resistor já existe: ligue só VCC, GND e dados).

Código (Uno)

02_dht11_uno.ino

```
/*
  02 - DHT11: temperatura e umidade do ar
  Placa: Arduino Uno/Nano     Biblioteca: "DHT sensor library" (Adafruit) + "Adafruit Unified Sensor"

  O DHT11 aparece de 3 jeitos - CONFIRA A SERIGRAFIA da sua peca antes de ligar:
    a) sensor "pelado" azul, 4 pinos:  VCC | DATA | (nao ligado) | GND   -> precisa de resistor 10k entre
DATA e VCC
    b) modulo 3 pinos "VCC GND OUT" (ou "+ - OUT"):  ja tem o resistor na placa
    c) modulo 3 pinos "S  -  (meio)" tipo KY-015: S = dados, "-" = GND e o pino do MEIO (sem nome) = VCC
       Nos modulos, a ORDEM muda de fabricante para fabricante: leia a placa, nao decore a posicao!
  Neste sketch: DATA/OUT/S -> pino 2, VCC -> 5V, GND -> GND.

  Fisica: a umidade e medida por um POLIMERO que absorve agua e muda sua CAPACITANCIA; a
  temperatura, por um TERMISTOR NTC (resistencia cai com o calor). Um chip converte os dois em
  numeros e manda pelo unico fio de dados, 40 bits por leitura. Limite: 1 leitura a cada 2 s.
*/
#include <DHT.h>

const int PINO_DHT = 2;
DHT dht(PINO_DHT, DHT11);          // troque DHT11 por DHT22 se usar o branco (mais preciso)

void setup() {
  Serial.begin(9600);
  dht.begin();
  Serial.println("umidade_%,temperatura_C,sensacao_C");
}

void loop() {
  delay(2000);                                    // o DHT11 nao aguenta ler mais rapido
  float h = dht.readHumidity();
  float t = dht.readTemperature();                // Celsius
  if (isnan(h) || isnan(t)) {                     // NaN = "not a number": leitura falhou
    Serial.println("falha na leitura (fio de dados? pull-up? pino?)");
    return;
  }
  float sensacao = dht.computeHeatIndex(t, h, false);   // indice de calor (como o corpo sente)
  Serial.print(h, 0); Serial.print(",");
  Serial.print(t, 1); Serial.print(",");
  Serial.println(sensacao, 1);
}
```

- `isnan()` detecta leitura falha — o DHT devolve *NaN* quando o protocolo não fecha (fio ruim, sem pull-up, ou leitura antes de 2 s).

- `computeHeatIndex()` calcula o **índice de calor**: a temperatura que o corpo sente combinando calor e umidade (a 32 °C com 80 % de umidade a sensação é de 41 °C).
- O `delay(2000)` é limite do sensor; no ESP32 usamos `millis()` para o Wi-Fi continuar respondendo.

Calibração

1. **Umidade 75 %:** pote fechado com sal de cozinha úmido (pasta de sal + pouca água) por 1 h: o ar dentro fica em **75,3 % UR** a 25 °C. **33 %:** cloreto de magnésio (sal do mar/suplemento). Compare e anote o desvio.
2. **Temperatura:** compare com o DS18B20 da Aula 1 no mesmo ambiente — espere ±2 °C.
3. O DHT11 não tem ajuste interno: corrija por software (`h = h + desvio`). Se o desvio passar de 10 %, o sensor está gasto (o polímero envelhece).

Monitor Wi-Fi no celular (ESP32)

O sketch do ESP32 cria a rede **Sensor-DHT11** (senha `12345678`). Conecte o celular a ela (desligue os dados móveis) e abra **http://192.168.4.1**: a página mostra o valor principal em destaque, os demais valores em cartões, um gráfico das últimas 120 leituras e um aviso de **ALERTA** quando o limite do código é ultrapassado.

- A página é a mesma para todos os sensores (`pagina_web.h`): ela pede `/dados` a cada 2 s e o ESP32 responde um **JSON** com uma lista de itens `{n, v, u}` (nome, valor, unidade).
- O servidor está em `monitor_wifi.h`: `monitorIniciar(nome)`, `monitorAtender()` no `loop()` e `monitorPublicar(json)` quando há leitura nova. Para um sensor novo, só muda o que entra no JSON.
- O sketch calcula também o **ponto de orvalho** (fórmula de Magnus): a temperatura em que o ar saturaria e a água condensaria — se uma superfície estiver abaixo dele, ela "sua" (mofo em paredes!).

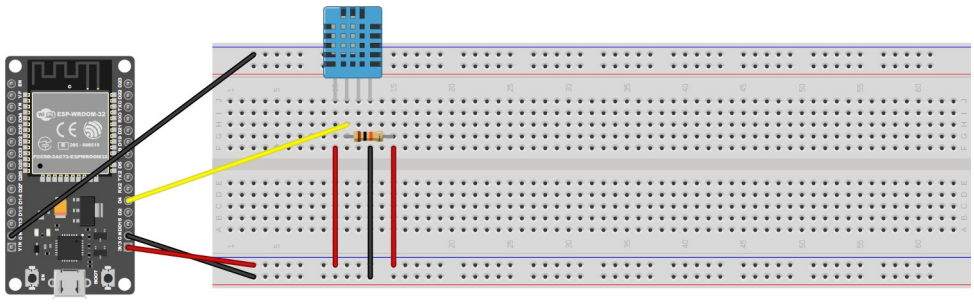


Figura 4 — Aula 2 — DHT11 no ESP32 (3V3, GPIO 4).

02b_dht11_esp32_web.ino

```
/*
  02b - DHT11 no ESP32 com monitor Wi-Fi
  Placa: ESP32 Dev Module     Biblioteca: DHT sensor library (Adafruit)
  Arquivos extras: monitor_wifi.h, pagina_web.h

  Ligacoes: VCC -> 3V3 (o DHT11 funciona de 3 a 5,5 V)   GND -> GND   DATA/OUT/S -> GPIO 4
            (sensor de 4 pinos: resistor 10 k entre DATA e 3V3; modulos ja tem)
  Celular: rede "Sensor-DHT11" -> http://192.168.4.1
*/
#include <DHT.h>
#include "monitor_wifi.h"

const int PINO_DHT = 4;
```

```
DHT dht(PINO_DHT, DHT11);
unsigned long ultimaLeitura = 0;
const float UMID_ALERTA = 70, TEMP_ALERTA = 30;

void setup() {
  Serial.begin(115200);
  dht.begin();
  monitorIniciar("Sensor-DHT11");
}

void loop() {
  monitorAtender();
  if (millis() - ultimaLeitura >= 2000) {          // limite do DHT11
    ultimaLeitura = millis();
    float h = dht.readHumidity(), t = dht.readTemperature();
    if (isnan(h) || isnan(t)) { monitorPublicar(jsonMonitor("DHT11", item("Temperatura", 0, "C"), true,
"falha na leitura")); return; }
    float sensacao = dht.computeHeatIndex(t, h, false);
    // ponto de orvalho (aproximacao de Magnus): temperatura em que o ar satura
    float a = 17.27, b = 237.7, alpha = (a * t) / (b + t) + log(h / 100.0);
    float orvalho = (b * alpha) / (a - alpha);
    bool alerta = h > UMID_ALERTA || t > TEMP_ALERTA;
    Serial.printf("%.0f %%  %.1f C\n", h, t);
    monitorPublicar(jsonMonitor("DHT11 - temperatura e umidade",
      item("Temperatura", t, "C") + "," + item("Umidade", h, "%") + "," + item("Sensacao", sensacao, "C")
+ "," + item("Pto. orvalho", orvalho, "C"),
      alerta, h > UMID_ALERTA ? "umidade alta (mofo!)" : "temperatura alta"));
  }
}
```

Experimente

1. Sopre no sensor (sem encostar): umidade e temperatura sobem? Quanto tempo levam para voltar? (Resposta lenta: ~10 s.)
2. Coloque o sensor num saco com um pano molhado; depois perto de um secador (frio). Compare os extremos.
3. Tente ler a cada 500 ms: o que a biblioteca devolve?

Desafios (com alerta por LED RGB e buzzer)

1. **2.1 Alerta de mofo** — umidade > 70 % por mais de 5 min: LED amarelo; > 80 %: vermelho + buzzer a cada 10 s (histerese de 3 %).
2. **2.2 Conforto térmico** — LED verde quando 20–26 °C **e** 40–60 % UR; azul (frio), vermelho (calor) ou roxo (úmido) fora disso. Tabela-verdade primeiro!
3. **2.3 Estufa** — botão define a temperatura-alvo (cada toque +1 °C); buzzer bipa se sair da faixa ±2 °C; LED mostra se está abaixo (azul) ou acima (vermelho).
4. **2.4 Ponto de orvalho na parede** — segundo sensor (DS18B20) encostado numa superfície: se a temperatura dela ficar abaixo do ponto de orvalho do ar, alerta de condensação.
5. **2.5 Dois sensores no celular** — mostre DHT11 **e** DS18B20 na mesma página (basta acrescentar itens ao JSON).

Aula 3 — MQ-2: gás inflamável e fumaça

O que é e para que serve

O **MQ-2** detecta **GLP (gás de cozinha), butano, propano, metano, hidrogênio, álcool, CO e fumaça**, de ~300 a 10 000 ppm. É o sensor dos detectores de vazamento de gás domésticos e de alarmes de incêndio simples. Vem num módulo de 4 pinos com saída **analógica (AO)** e **digital (DO)** com limiar ajustável por potenciômetro.

A química: um semicondutor que "cheira"

Dentro da malha metálica há um tubinho de cerâmica coberto por **dióxido de estanho (SnO_2)**, um semicondutor, com um **filamento aquecedor** dentro que o mantém a ~300 °C (por isso o módulo esquenta e consome 150 mA). No ar limpo, moléculas de **oxigênio** se prendem à superfície do SnO_2 e capturam elétrons dele — a resistência do material fica **alta**. Quando um **gás redutor** (GLP, CO, H_2…) chega, ele reage com esse oxigênio adsorvido (queima na superfície), **devolve os elétrons** e a resistência **cai**. Quanto mais gás, menor a resistência. Medimos essa resistência **Rs** com um divisor de tensão (o resistor de carga RL ≈ 5 kΩ está no módulo) — é o mesmo princípio do divisor que já usamos, agora com um resistor variável.

O datasheet fornece um gráfico **log-log** da razão **Rs/R0** contra a concentração em ppm, uma reta para cada gás: ppm = a · (Rs/R0)^b. **R0 é a resistência no ar limpo** — e varia de sensor para sensor, com a temperatura e com a idade. Por isso todo MQ precisa de **calibração de R0** no ar limpo: o datasheet diz que Rs/R0 = 9,83 nessa condição para o MQ-2. A biblioteca `MQUnifiedsensor` guarda os coeficientes (a, b) de cada gás e faz as contas.

> **Pré-aquecimento e limitações**
>
> O datasheet pede **24 h de aquecimento na primeira vez** (o SnO_2 precisa "assentar") e uns **3 min** nas vezes seguintes. Sem isso a leitura vai caindo sozinha nos primeiros minutos.
>
> O MQ-2 **não distingue** gases: ele responde a todos ao mesmo tempo. O valor "ppm de GLP" só faz sentido se você souber que é GLP. Umidade e temperatura também mudam Rs (±20 %). É um sensor de **alarme**, não de laboratório.

Montagem

Módulo MQ-2	Arduino Uno	ESP32
VCC	5V	**VIN** (5 V) — nunca 3V3
GND	GND	GND
AO	A0	1 kΩ → **GPIO 34** → 2 kΩ → GND (divisor)
DO (opcional)	pino 3	GPIO 27 (sai em 5 V: use outro divisor ou só o AO)

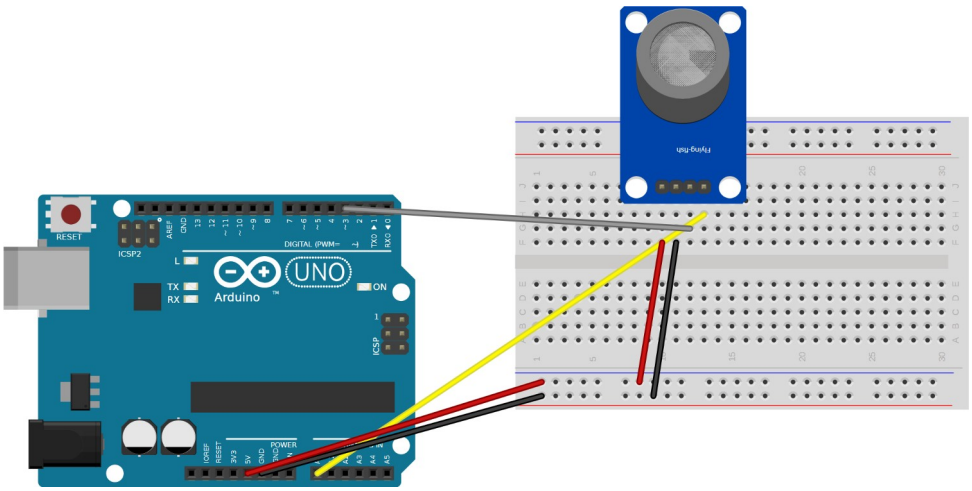


Figura 5 — Aula 3 — módulo MQ-2 no Uno: AO em A0, DO no pino 3.

Código (Uno)

03_mq2_uno.ino

```
/*
  03 - MQ-2: gas inflamavel (GLP, butano, propano, hidrogenio) e fumaca
  Placa: Arduino Uno/Nano     Biblioteca: MQUnifiedsensor (Miguel Califa)

  Modulo MQ-2 (4 pinos):  VCC -> 5V   GND -> GND   AO -> A0   DO -> (opcional) pino 3
  O modulo tem um potenciometro que so ajusta o limiar do pino DIGITAL (DO); a saida analogica
  AO nao depende dele.

  Quimica: dentro do sensor ha uma ceramica de OXIDO DE ESTANHO (SnO2) AQUECIDA (~300 C) por um
  filamento. No ar limpo, oxigenio se prende a superficie e "rouba" eletrons: a resistencia fica
  ALTA. Gases redutores (GLP, CO, H2, fumaca) reagem com esse oxigenio, devolvem os eletrons e a
  resistencia CAI. Medimos a resistencia Rs por um divisor de tensao (RL = 5 kOhm no modulo).
  A relacao Rs/R0 x concentracao (ppm) e uma reta em escala log-log (datasheet) - a biblioteca
  guarda os coeficientes (a, b) de cada gas:  ppm = a * (Rs/R0)^b
  R0 = resistencia no AR LIMPO: por isso precisamos CALIBRAR (Rs/R0 = 9,83 no ar limpo, pelo datasheet).
  Pre-aquecimento: o datasheet pede 24 h na primeira vez e alguns minutos nas seguintes.
*/
#include <MQUnifiedsensor.h>

#define PLACA "Arduino UNO"
#define PINO_AO A0
#define TIPO "MQ-2"
#define TENSAO 5
#define BITS_ADC 10
#define RAZAO_AR_LIMPO 9.83
const int PINO_DO = 3;

MQUnifiedsensor MQ2(PLACA, TENSAO, BITS_ADC, PINO_AO, TIPO);

void setup() {
  Serial.begin(9600);
  pinMode(PINO_DO, INPUT);
  MQ2.setRegressionMethod(1);          // ppm = a * ratio^b
  MQ2.init();

  Serial.println("Calibrando no AR LIMPO (afaste gases, espere o pre-aquecimento)...");
  float r0 = 0;
  for (int i = 0; i < 10; i++) { MQ2.update(); r0 += MQ2.calibrate(RAZAO_AR_LIMPO); Serial.print(".");
delay(100); }
  MQ2.setR0(r0 / 10);
  Serial.print("\nR0 = "); Serial.print(r0 / 10, 2); Serial.println(" kOhm");
  if (isinf(r0)) Serial.println("ERRO: R0 infinito - fio aberto?");
  if (r0 == 0)   Serial.println("ERRO: R0 zero - curto?");
  Serial.println("GLP_ppm,CO_ppm,fumaca_ppm,Rs/R0,DO");
}

float ler(float a, float b) { MQ2.setA(a); MQ2.setB(b); return MQ2.readSensor(); }
```

```
void loop() {
  MQ2.update();                                  // le o A0 uma vez
  float glp    = ler(574.25, -2.222);            // coeficientes do datasheet (biblioteca)
  float co     = ler(36974,  -3.109);
  float fumaca = ler(30000000, -8.308);
  float razao  = MQ2.readSensorR0Rs() > 0 ? 1.0 / MQ2.readSensorR0Rs() : 0;   // Rs/R0

  Serial.print(glp, 0);    Serial.print(",");
  Serial.print(co, 0);     Serial.print(",");
  Serial.print(fumaca, 0); Serial.print(",");
  Serial.print(razao, 2);  Serial.print(",");
  Serial.println(digitalRead(PINO_DO) == LOW ? "ALERTA" : "ok");   // DO vai a LOW acima do limiar do pot
  delay(500);
}
```

- `MQ2.calibrate(9.83)` lê Rs e devolve **R0 = Rs / 9,83**; fazemos 10 vezes e tiramos a média. Se der `inf` ou 0, há fio solto ou curto.
- `setA/setB` + `readSensor()` aplicam a reta de cada gás; a função `ler(a, b)` evita repetir código.
- `readSensorR0Rs()` devolve R0/Rs; invertemos para mostrar **Rs/R0** — o número "cru" que não depende de nenhuma curva: no ar limpo ≈ 9,8; com gás cai para 1–3.
- O pino **DO** vai a LOW quando AO passa do limiar do potenciômetro: um comparador de hardware, útil sem microcontrolador.

Calibração passo a passo

1. Ligue e espere ≥ 3 min (ideal: deixe ligado na véspera).
2. Leve para um lugar com **ar limpo** (janela aberta, longe de álcool, gás e da própria respiração).
3. Reinicie o Arduino: o `setup()` calibra e imprime R0. Anote-o. Se for repetir muitas vezes, grave R0 na EEPROM em vez de recalibrar toda vez (desafio 3.4).
4. Teste: isqueiro sem acender a 5 cm por 2 s → GLP deve subir para milhares de ppm e Rs/R0 cair. Álcool num algodão a 10 cm também responde.
5. Ajuste o **potenciômetro do módulo** até o LED do DO acender só quando você quiser (ponto de alarme).

Monitor Wi-Fi no celular (ESP32)

O sketch do ESP32 cria a rede **Sensor-MQ2** (senha `12345678`). Conecte o celular a ela (desligue os dados móveis) e abra **http://192.168.4.1**: a página mostra o valor principal em destaque, os demais valores em cartões, um gráfico das últimas 120 leituras e um aviso de **ALERTA** quando o limite do código é ultrapassado.

- A página é a mesma para todos os sensores (`pagina_web.h`): ela pede `/dados` a cada 2 s e o ESP32 responde um **JSON** com uma lista de itens `{n, v, u}` (nome, valor, unidade).
- O servidor está em `monitor_wifi.h`: `monitorIniciar(nome)`, `monitorAtender()` no `loop()` e `monitorPublicar(json)` quando há leitura nova. Para um sensor novo, só muda o que entra no JSON.
- No ESP32 o módulo continua em **5 V** (VIN) e o AO passa pelo **divisor 1 kΩ/2 kΩ**; a biblioteca é configurada para 3,3 V e 12 bits. Como calibramos R0 já com o divisor, a razão Rs/R0 continua correta.

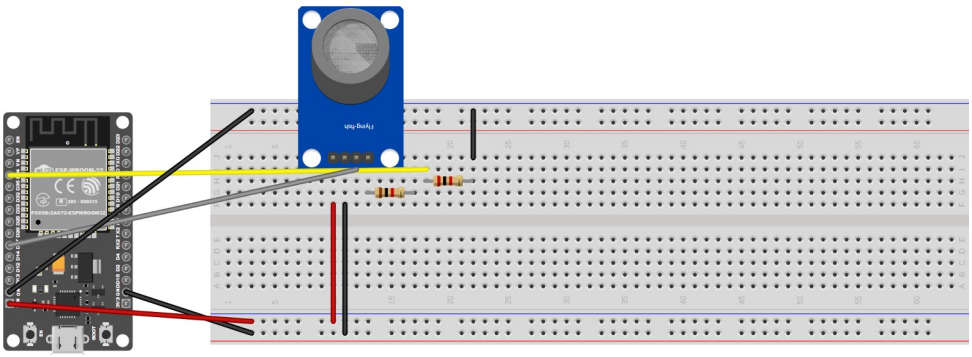


Figura 6 — Aula 3 — MQ-2 no ESP32: alimentação pelo VIN e divisor de tensão no AO.

03b_mq2_esp32_web.ino

```
/*
  03b - MQ-2 no ESP32 com monitor Wi-Fi
  Placa: ESP32 Dev Module     Biblioteca: MQUnifiedsensor
  Arquivos extras: monitor_wifi.h, pagina_web.h

  ATENCAO (fisica): o MQ-2 PRECISA de 5 V para o aquecedor (~150 mA) - alimente pelo VIN.
  A saida AO vai ate 5 V e o ESP32 so aguenta 3,3 V: use um DIVISOR 1 k / 2 k (AO -> 1k -> GPIO34 -> 2k ->
GND).
  Com o divisor, o ESP32 le 2/3 da tensao real; a biblioteca e configurada para 3,3 V e 12 bits e
  corrigimos o fator 3/2 na leitura (veja FATOR_DIVISOR).
  Ligacoes: VCC -> VIN   GND -> GND   AO -> divisor -> GPIO 34   DO -> GPIO 27 (opcional)
  Celular: rede "Sensor-MQ2" -> http://192.168.4.1
*/
#include <MQUnifiedsensor.h>
#include "monitor_wifi.h"

#define PLACA "ESP-32"
#define PINO_AO 34
#define TIPO "MQ-2"
#define TENSAO 3.3
#define BITS_ADC 12
#define RAZAO_AR_LIMPO 9.83
const int PINO_DO = 27;
const float FATOR_DIVISOR = 1.5;     // 1k/2k: o GPIO ve 2/3 da tensao; nao usado pela lib, so informativo
const float GLP_ALERTA = 1000;       // ppm

MQUnifiedsensor MQ2(PLACA, TENSAO, BITS_ADC, PINO_AO, TIPO);
unsigned long ultimaLeitura = 0;

float ler(float a, float b) { MQ2.setA(a); MQ2.setB(b); return MQ2.readSensor(); }

void setup() {
  Serial.begin(115200);
  pinMode(PINO_DO, INPUT);
  analogReadResolution(12);
  MQ2.setRegressionMethod(1);
  MQ2.init();
  Serial.println("Calibrando no ar limpo...");
  float r0 = 0;
  for (int i = 0; i < 10; i++) { MQ2.update(); r0 += MQ2.calibrate(RAZAO_AR_LIMPO); delay(100); }
  MQ2.setR0(r0 / 10);
  Serial.printf("R0 = %.2f kOhm\n", r0 / 10);
  monitorIniciar("Sensor-MQ2");
}

void loop() {
  monitorAtender();
  if (millis() - ultimaLeitura >= 1000) {
    ultimaLeitura = millis();
    MQ2.update();
    float glp = ler(574.25, -2.222), co = ler(36974, -3.109), fumaca = ler(30000000, -8.308);
    float razao = MQ2.readSensorR0Rs() > 0 ? 1.0 / MQ2.readSensorR0Rs() : 0;
    bool alerta = glp > GLP_ALERTA || digitalRead(PINO_DO) == LOW;
    Serial.printf("GLP %.0f ppm  CO %.0f  fumaca %.0f  Rs/R0 %.2f\n", glp, co, fumaca, razao);
    monitorPublicar(jsonMonitor("MQ-2 - gas inflamavel",
```

```
      item("GLP", glp, "ppm") + "," + item("CO", co, "ppm") + "," + item("Fumaca", fumaca, "ppm") + "," +
item("Rs/R0", razao, ""),
      alerta, "gas detectado!"));
  }
}
```

Experimente

1. Observe Rs/R0 cair nos 3 primeiros minutos após ligar (pré-aquecimento). Quando estabiliza?
2. Compare a resposta ao isqueiro e ao álcool: qual sobe mais rápido? Qual demora mais para voltar?
3. Sopre no sensor: por que a leitura muda? (umidade + CO_2 do hálito + esfriamento do aquecedor)

Desafios (com alerta por LED RGB e buzzer)

1. **3.1 Detector de vazamento** — GLP > 1000 ppm por 3 s seguidos: buzzer contínuo + LED vermelho piscando; volta ao normal só quando ficar < 500 ppm por 10 s (histerese no tempo e no valor).
2. **3.2 Três níveis** — verde/amarelo/vermelho e bipe cada vez mais rápido, como o alarme de ré, mas com ppm.
3. **3.3 Botão "silenciar por 1 min"** e LED indicando que o alarme está silenciado (pisca lento).
4. **3.4 R0 na EEPROM** — comando `c` no Serial recalibra e grava R0; ao ligar, lê da EEPROM (sem esperar o ar limpo toda vez).
5. **3.5 Alarme de fumaça no celular** — com o ESP32, mostre na página o **tempo desde o último alarme** e o **pico** de ppm do dia.

Aula 4 — MQ-135: qualidade do ar

O que é e para que serve

O **MQ-135** é da mesma família do MQ-2, mas o SnO_2 é **dopado** para responder mais a **amônia (NH_3), óxidos de nitrogênio, álcool, benzeno, fumaça e CO_2**. É vendido como "sensor de qualidade do ar" para salas, escritórios, galinheiros e monitoramento de CO_2 em ambientes fechados — onde a concentração de CO_2 indica se a ventilação é suficiente (ar externo ≈ 420 ppm; acima de 1000 ppm dá sonolência; acima de 2000, dor de cabeça).

A química: o mesmo princípio, outra "receita"

Vale tudo o que foi dito do MQ-2: SnO_2 aquecido, oxigênio adsorvido, resistência cai com gases redutores. A diferença está nos **aditivos** (dopantes) que mudam quais moléculas reagem melhor na superfície. **Atenção:** o CO_2 não é um gás redutor forte — o MQ-135 "sente" CO_2 mal e de forma indireta. Sensores de CO_2 **de verdade** usam **NDIR** (infravermelho: o CO_2 absorve luz de 4,26 µm) e custam 10× mais. Por isso o código soma **400 ppm** (a base atmosférica) ao valor da curva e chamamos de **CO_2 estimado**. Para a aula, ele mostra muito bem a *tendência*: a sala enchendo de gente, a janela abrindo.

Montagem

Idêntica à do MQ-2 (mesmo módulo de 4 pinos): VCC → 5 V / VIN, GND, AO → A0 (Uno) ou divisor → GPIO 34 (ESP32), DO → pino 3 / GPIO 27.

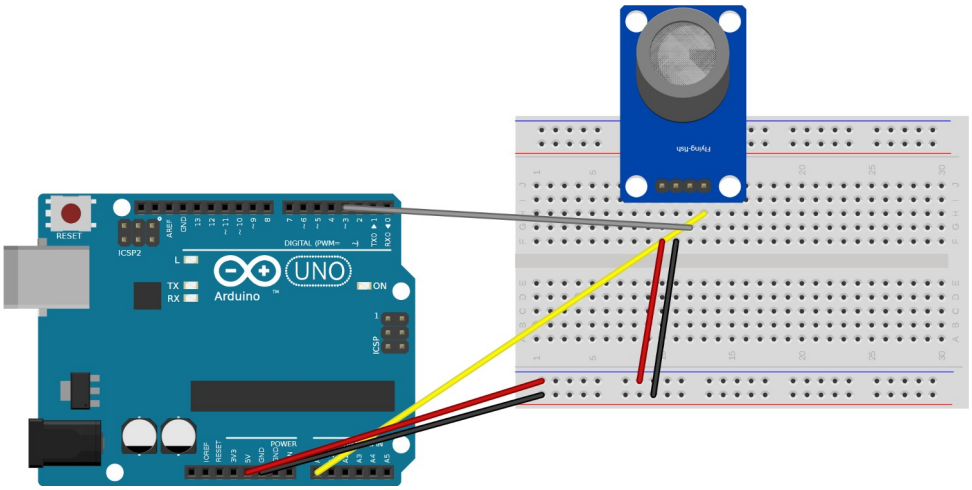


Figura 7 — Aula 4 — MQ-135 no Uno (mesma ligação do MQ-2).

Código (Uno)

04_mq135_uno.ino

```
/*
  04 - MQ-135: qualidade do ar (CO2 estimado, amonia, alcool, benzeno, fumaca)
  Placa: Arduino Uno/Nano     Biblioteca: MQUnifiedsensor

  Modulo MQ-135:  VCC -> 5V   GND -> GND   AO -> A0   DO -> (opcional) pino 3

  Mesmo principio do MQ-2 (SnO2 aquecido), mas o material foi "dopado" para reagir mais a
  amonia (NH3), oxidos de nitrogenio, alcool, benzeno, fumaca e CO2. IMPORTANTE: o MQ-135 NAO e
  um sensor de CO2 de verdade (isso seria um NDIR, infravermelho); ele ESTIMA CO2 pela curva do
  datasheet, e o ar limpo externo tem ~400-420 ppm. Por isso somamos 400 ao valor da curva.
  Rs/R0 no ar limpo = 3,6. Calibre ao ar livre (ou janela aberta), apos pre-aquecer.
*/
```

```
#include <MQUnifiedsensor.h>

#define PLACA "Arduino UNO"
#define PINO_AO A0
#define TIPO "MQ-135"
#define TENSAO 5
#define BITS_ADC 10
#define RAZAO_AR_LIMPO 3.6
const int PINO_DO = 3;

MQUnifiedsensor MQ135(PLACA, TENSAO, BITS_ADC, PINO_AO, TIPO);

void setup() {
  Serial.begin(9600);
  pinMode(PINO_DO, INPUT);
  MQ135.setRegressionMethod(1);
  MQ135.init();
  Serial.println("Calibrando no AR LIMPO...");
  float r0 = 0;
  for (int i = 0; i < 10; i++) { MQ135.update(); r0 += MQ135.calibrate(RAZAO_AR_LIMPO); Serial.print(".");
delay(100); }
  MQ135.setR0(r0 / 10);
  Serial.print("\nR0 = "); Serial.print(r0 / 10, 2); Serial.println(" kOhm");
  Serial.println("CO2_ppm,NH4_ppm,alcool_ppm,CO_ppm,Rs/R0");
}

float ler(float a, float b) { MQ135.setA(a); MQ135.setB(b); return MQ135.readSensor(); }

void loop() {
  MQ135.update();
  float co2    = ler(110.47, -2.862) + 400;      // + 400 ppm de base atmosferica
  float nh4    = ler(102.2,  -2.473);
  float alcool = ler(77.255, -3.18);
  float co     = ler(605.18, -3.937);
  float razao  = MQ135.readSensorR0Rs() > 0 ? 1.0 / MQ135.readSensorR0Rs() : 0;

  Serial.print(co2, 0);    Serial.print(",");
  Serial.print(nh4, 1);    Serial.print(",");
  Serial.print(alcool, 1); Serial.print(",");
  Serial.print(co, 1);     Serial.print(",");
  Serial.println(razao, 2);
  delay(1000);
}
```

- Rs/R0 no ar limpo = **3,6** para o MQ-135 (era 9,83 no MQ-2) — cada modelo tem o seu, sempre do datasheet.
- Coeficientes: CO_2 (110,47; −2,862), NH_4 (102,2; −2,473), álcool (77,255; −3,18), CO (605,18; −3,937).
- Uma **escala de qualidade** (boa < 800, razoável < 1200, ruim < 2000 ppm) transforma número em decisão — é o que um produto faz.

Calibração

1. Igual ao MQ-2, mas o “ar limpo” precisa ser **ar externo** (dentro da sala já há 600–1500 ppm de CO_2 com gente).
2. Pré-aqueça ≥ 3 min (24 h na primeira vez).
3. Teste: respire perto (CO_2 + umidade), álcool em gel na mão a 20 cm (álcool), amoníaco de limpeza **com a tampa** aberta a 30 cm (NH_3).

Monitor Wi-Fi no celular (ESP32)

O sketch do ESP32 cria a rede **Sensor-MQ135** (senha `12345678`). Conecte o celular a ela (desligue os dados móveis) e abra **http://192.168.4.1**: a página mostra o valor principal em destaque, os demais valores em cartões, um gráfico das últimas 120 leituras e um aviso de **ALERTA** quando o limite do código é ultrapassado.

- A página é a mesma para todos os sensores (`pagina_web.h`): ela pede `/dados` a cada 2 s e o ESP32 responde um **JSON** com uma lista de itens `{n, v, u}` (nome, valor, unidade).
- O servidor está em `monitor_wifi.h`: `monitorIniciar(nome)`, `monitorAtender()` no `loop()` e `monitorPublicar(json)` quando há leitura nova. Para um sensor novo, só muda o que entra no JSON.
- A página mostra o CO_2 estimado em destaque e a mensagem de alerta traz a **classificação** (boa/razoável/ruim/péssima).

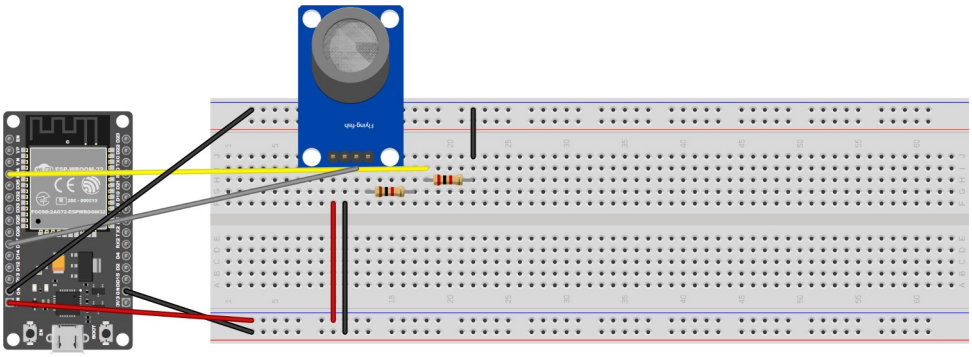


Figura 8 — Aula 4 — MQ-135 no ESP32.

04b_mq135_esp32_web.ino

```
/*
  04b - MQ-135 no ESP32 com monitor Wi-Fi (qualidade do ar)
  Placa: ESP32 Dev Module     Biblioteca: MQUnifiedsensor
  Arquivos extras: monitor_wifi.h, pagina_web.h

  Ligacoes (iguais ao MQ-2): VCC -> VIN (5 V)   GND -> GND   AO -> 1k -> GPIO 34 -> 2k -> GND   DO -> GPIO
27
  Celular: rede "Sensor-MQ135" -> http://192.168.4.1
*/
#include <MQUnifiedsensor.h>
#include "monitor_wifi.h"

#define PLACA "ESP-32"
#define PINO_AO 34
#define TIPO "MQ-135"
#define TENSAO 3.3
#define BITS_ADC 12
#define RAZAO_AR_LIMPO 3.6
const int PINO_DO = 27;
const float CO2_ALERTA = 1500;       // ppm: acima disso a sala precisa ventilar

MQUnifiedsensor MQ135(PLACA, TENSAO, BITS_ADC, PINO_AO, TIPO);
unsigned long ultimaLeitura = 0;

float ler(float a, float b) { MQ135.setA(a); MQ135.setB(b); return MQ135.readSensor(); }

void setup() {
  Serial.begin(115200);
  pinMode(PINO_DO, INPUT);
  analogReadResolution(12);
  MQ135.setRegressionMethod(1);
  MQ135.init();
  Serial.println("Calibrando no ar limpo (ao ar livre)...");
```

```
  float r0 = 0;
  for (int i = 0; i < 10; i++) { MQ135.update(); r0 += MQ135.calibrate(RAZAO_AR_LIMPO); delay(100); }
  MQ135.setR0(r0 / 10);
  Serial.printf("R0 = %.2f kOhm\n", r0 / 10);
  monitorIniciar("Sensor-MQ135");
}

void loop() {
  monitorAtender();
  if (millis() - ultimaLeitura >= 1000) {
    ultimaLeitura = millis();
    MQ135.update();
    float co2 = ler(110.47, -2.862) + 400, nh4 = ler(102.2, -2.473), alcool = ler(77.255, -3.18), co =
ler(605.18, -3.937);
    float razao = MQ135.readSensorR0Rs() > 0 ? 1.0 / MQ135.readSensorR0Rs() : 0;
    const char* qualidade = co2 < 800 ? "boa" : co2 < 1200 ? "razoavel" : co2 < 2000 ? "ruim" : "pessima";
    bool alerta = co2 > CO2_ALERTA;
    Serial.printf("CO2 ~%.0f ppm (%s)  NH4 %.1f  alcool %.1f\n", co2, qualidade, nh4, alcool);
    monitorPublicar(jsonMonitor(String("MQ-135 - ar ").c_str(),
      item("CO2 estimado", co2, "ppm") + "," + item("Amonia", nh4, "ppm") + "," + item("Alcool", alcool,
"ppm") + "," + item("CO", co, "ppm") + "," + item("Rs/R0", razao, ""),
      alerta, "ventile a sala: " + String(qualidade)));
  }
}
```

Comparando MQ-2 e MQ-135

	MQ-2	**MQ-135**
Alvo principal	GLP, fumaça, H_2, CO	NH_3, NOx, álcool, benzeno, CO_2 (estimado)
Rs/R0 no ar limpo	9,83	3,6
Uso típico	vazamento de gás, incêndio	qualidade do ar interno
Responde a hálito?	pouco	muito (CO_2 + umidade)
Preço / consumo	≈ iguais (~R$ 15, 150 mA)	

Experimente

1. Feche a sala com a turma por 20 min com o sensor ligado: quanto o CO_2 estimado sobe? Abra a janela e cronometre a queda.
2. Ponha os dois sensores (MQ-2 e MQ-135) lado a lado e exponha a álcool: qual reage mais?

Desafios (com alerta por LED RGB e buzzer)

1. **4.1 Semáforo da ventilação** — verde < 800, amarelo < 1200, vermelho ≥ 1200 ppm; acima de 2000 o buzzer toca 3 bipes a cada minuto até melhorar.
2. **4.2 Média de 1 minuto** — o alerta usa a média dos últimos 60 s, não a leitura instantânea (evita disparar com uma respiração perto).
3. **4.3 Tendência** — LED pisca rápido se o CO_2 está **subindo** mais de 100 ppm/min, lento se descendo (compare com a média de 1 min atrás).
4. **4.4 Índice combinado** — junte MQ-135 + DHT11: “ar ruim” se CO_2 > 1200 **ou** umidade > 75 %; mostre os dois no celular.

5. **4.5 Registro** — mande as leituras pela serial para o `plot_distancia.py` (aula anterior) adaptado, e plote CO_2 × tempo de uma aula inteira.

Aula 5 — BME680: estação meteorológica num chip

O que é e para que serve

O **BME680** (Bosch) tem, num chip de 3 × 3 mm, **quatro sensores**: temperatura (±1 °C), umidade (±3 %), **pressão barométrica** (±1 hPa, de 300 a 1100 hPa) e um sensor de **gás** (VOC — compostos orgânicos voláteis). Comunica-se por **I²C** ou SPI. Serve para estações meteorológicas, altímetros (drones, mochilas), monitoramento de qualidade do ar interno (é o sensor que muitos purificadores usam) e previsão de tempo simples (pressão caindo = chuva chegando).

A física: quatro sensores MEMS

- **Pressão:** uma membrana de silício microscópica se deforma com a pressão do ar; resistores **piezorresistivos** nela mudam de valor. A pressão cai **~12 hPa a cada 100 m** de altitude perto do nível do mar — daí a **altitude barométrica**: h = 44330 · (1 − (p/p_o)^0,19), com p_o a pressão ao nível do mar (1013,25 hPa padrão; use o valor real do dia para precisão).
- **Umidade:** polímero capacitivo, como no DHT, mas com calibração de fábrica muito melhor e resposta em 8 s.
- **Temperatura:** diodo de silício (o mesmo princípio do DS18B20) — note que o próprio chip se aquece um pouco, e ainda mais com o aquecedor de gás ligado (+1 a 2 °C).
- **Gás:** uma placa de **óxido metálico aquecida a 320 °C** — exatamente o princípio do MQ, em miniatura e com aquecimento por pulsos de 150 ms. Ele entrega a **resistência em kΩ**: cai quando há VOC (álcool, tinta, perfume, hálito, solvente). Não dá ppm de um gás específico; a Bosch calcula um "IAQ" com um algoritmo proprietário; nós fazemos um **índice relativo**: 100 % = o ar mais limpo já visto.

I²C usa dois fios: **SDA** (dados) e **SCL** (relógio). Cada chip tem um **endereço** (0x76 ou 0x77, escolhido pelo pino SDO): vários sensores compartilham os mesmos dois fios. No Uno, SDA = A4 e SCL = A5; no ESP32, GPIO 21 e 22.

Montagem

BME680 (módulo)	Arduino Uno	ESP32
VCC / VIN	5V (Adafruit e módulos com regulador) ou 3,3 V (módulos "pelados": confira!)	3V3
GND	GND	GND
SCL / SCK	A5	GPIO 22
SDA / SDI	A4	GPIO 21
SDO (endereço)	solto = 0x77; GND = 0x76	idem

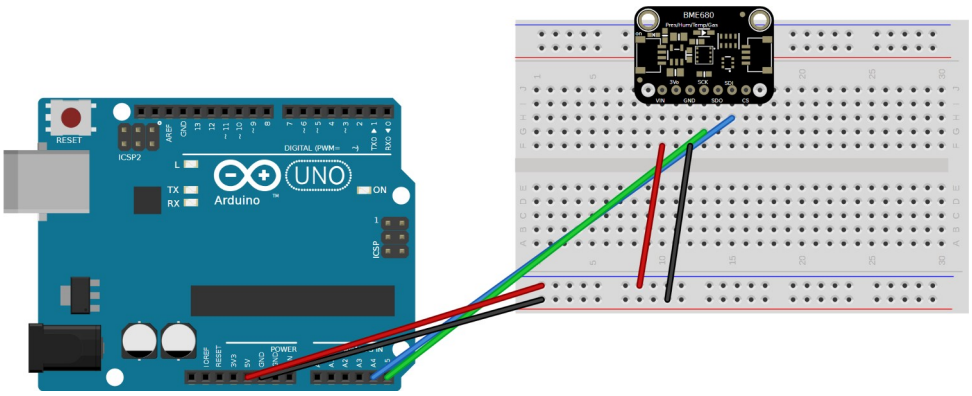


Figura 9 — Aula 5 — BME680 (I²C) no Uno: SDA em A4, SCL em A5.

Código (Uno)

05_bme680_uno.ino

```
/*
  05 - BME680: temperatura, umidade, pressao e gas (VOC) num so chip (Bosch)
  Placa: Arduino Uno/Nano     Bibliotecas: Adafruit BME680 Library + Adafruit Unified Sensor + Adafruit
BusIO

  Ligacoes (I2C):  VCC -> 3,3 V (a maioria dos modulos aceita 5 V, confira!)   GND -> GND
                   SDA -> A4        SCL -> A5        (SDO no GND = endereco 0x76; solto = 0x77)

  Fisica em 4 sensores num chip de 3 mm:
    - Pressao: uma membrana de silicio se deforma com a pressao (piezorresistivo). Com a pressao
      calculamos a ALTITUDE (a pressao cai ~12 hPa a cada 100 m perto do nivel do mar).
    - Umidade: polimero capacitivo, como no DHT, mas calibrado de fabrica (+-3 %).
    - Temperatura: diodo de silicio (o mesmo principio do DS18B20).
    - Gas: uma placa de oxido metalico AQUECIDA a 320 C (igual ao MQ, em miniatura). Le a
      RESISTENCIA em kOhm: cai quando ha compostos organicos volateis (VOC: alcool, tinta,
      perfume, halito...). Nao da ppm de um gas especifico; da um INDICE relativo de qualidade do ar.
  I2C: dois fios (SDA dados, SCL relogio) e cada chip tem um endereco - varios sensores no mesmo par.
*/
#include <Wire.h>
#include <Adafruit_Sensor.h>
#include <Adafruit_BME680.h>

#define PRESSAO_NIVEL_MAR 1013.25      // hPa; ajuste pelo boletim meteorologico do dia para altitude
exata
Adafruit_BME680 bme;                   // I2C

void setup() {
  Serial.begin(9600);
  if (!bme.begin(0x76) && !bme.begin(0x77)) {
    Serial.println("BME680 nao encontrado: confira SDA/SCL, alimentacao e endereco");
    while (1);
  }
  // quanto mais oversampling, menos ruido e mais tempo
  bme.setTemperatureOversampling(BME680_OS_8X);
  bme.setHumidityOversampling(BME680_OS_2X);
  bme.setPressureOversampling(BME680_OS_4X);
  bme.setIIRFilterSize(BME680_FILTER_SIZE_3);
  bme.setGasHeater(320, 150);          // 320 C por 150 ms
  Serial.println("temp_C,umid_%,pressao_hPa,gas_kOhm,altitude_m");
}

void loop() {
  if (!bme.performReading()) { Serial.println("falha na leitura"); return; }
  Serial.print(bme.temperature, 1);        Serial.print(",");
  Serial.print(bme.humidity, 0);           Serial.print(",");
  Serial.print(bme.pressure / 100.0, 1);   Serial.print(",");
  Serial.print(bme.gas_resistance / 1000.0, 1); Serial.print(",");
  Serial.println(bme.readAltitude(PRESSAO_NIVEL_MAR), 0);
  delay(2000);
}
```

- `bme.begin(0x76) || bme.begin(0x77)` tenta os dois endereços — o programa não trava por causa de um jumper.
- **Oversampling** (`OS_8X`): o chip mede 8 vezes e tira a média — menos ruído, mais tempo. O filtro IIR suaviza a pressão (que "treme" com o vento e portas).
- `setGasHeater(320, 150)`: 320 °C por 150 ms antes de cada leitura de gás; `performReading()` leva ~0,2 s.
- `readAltitude(p₀)`: mude `PRESSAO_NIVEL_MAR` para a pressão do dia (site do INMET/aeroporto) e veja a altitude bater com o mapa.

Calibração

1. **Pressão:** compare com a estação meteorológica mais próxima (INMET, aeroporto) corrigindo para a sua altitude; espere ±1–2 hPa.
2. **Temperatura:** compare com o DS18B20 — o BME680 costuma marcar 1–2 °C a mais por autoaquecimento; corrija por offset ou desligue o aquecedor de gás (`setGasHeater(0, 0)`) quando só quiser temperatura.
3. **Gás:** não há "ppm"; deixe o sensor rodar 30 min em ar limpo para o índice relativo ter uma referência boa (o sketch usa o **maior** valor de resistência visto).

Monitor Wi-Fi no celular (ESP32)

O sketch do ESP32 cria a rede **Sensor-BME680** (senha `12345678`). Conecte o celular a ela (desligue os dados móveis) e abra **http://192.168.4.1**: a página mostra o valor principal em destaque, os demais valores em cartões, um gráfico das últimas 120 leituras e um aviso de **ALERTA** quando o limite do código é ultrapassado.

- A página é a mesma para todos os sensores (`pagina_web.h`): ela pede `/dados` a cada 2 s e o ESP32 responde um **JSON** com uma lista de itens `{n, v, u}` (nome, valor, unidade).
- O servidor está em `monitor_wifi.h`: `monitorIniciar(nome)`, `monitorAtender()` no `loop()` e `monitorPublicar(json)` quando há leitura nova. Para um sensor novo, só muda o que entra no JSON.
- A página mostra 6 grandezas: temperatura, umidade, pressão, resistência de gás, índice de ar e altitude. A tendência da **pressão** no gráfico é a previsão do tempo: caindo rápido = chuva/vento; subindo = tempo firme.

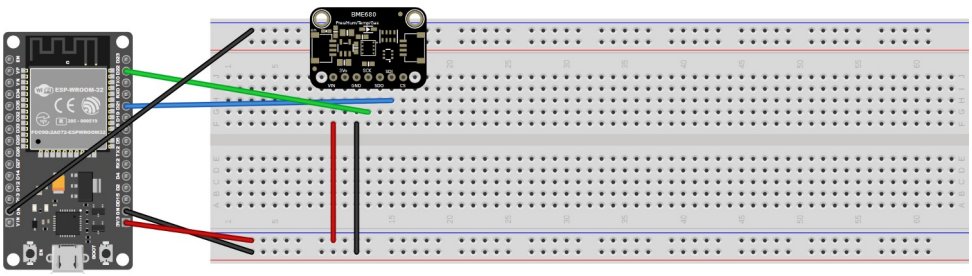


Figura 10 — Aula 5 — BME680 no ESP32 (3V3, SDA 21, SCL 22).

05b_bme680_esp32_web.ino

```
/*
  05b - BME680 no ESP32 com monitor Wi-Fi (estacao meteorologica de bolso)
  Placa: ESP32 Dev Module     Bibliotecas: Adafruit BME680, Adafruit Unified Sensor, Adafruit BusIO
```

```
  Arquivos extras: monitor_wifi.h, pagina_web.h

  Ligacoes (I2C): VCC -> 3V3   GND -> GND   SDA -> GPIO 21   SCL -> GPIO 22
  Celular: rede "Sensor-BME680" -> http://192.168.4.1
*/
#include <Wire.h>
#include <Adafruit_Sensor.h>
#include <Adafruit_BME680.h>
#include "monitor_wifi.h"

#define PRESSAO_NIVEL_MAR 1013.25
Adafruit_BME680 bme;
unsigned long ultimaLeitura = 0;
float gasLimpo = 0;                 // referencia de "ar limpo" para o indice

void setup() {
  Serial.begin(115200);
  Wire.begin(21, 22);
  if (!bme.begin(0x76) && !bme.begin(0x77)) { Serial.println("BME680 nao encontrado"); while (1)
delay(100); }
  bme.setTemperatureOversampling(BME680_OS_8X);
  bme.setHumidityOversampling(BME680_OS_2X);
  bme.setPressureOversampling(BME680_OS_4X);
  bme.setIIRFilterSize(BME680_FILTER_SIZE_3);
  bme.setGasHeater(320, 150);
  monitorIniciar("Sensor-BME680");
}

void loop() {
  monitorAtender();
  if (millis() - ultimaLeitura >= 3000) {
    ultimaLeitura = millis();
    if (!bme.performReading()) return;
    float t = bme.temperature, h = bme.humidity, p = bme.pressure / 100.0, gas = bme.gas_resistance /
1000.0;
    float alt = bme.readAltitude(PRESSAO_NIVEL_MAR);
    if (gas > gasLimpo) gasLimpo = gas;                        // o maior valor visto = ar mais limpo
    float indiceAr = gasLimpo > 0 ? 100.0 * gas / gasLimpo : 0; // 100 = igual ao melhor ar visto; menor =
mais VOC
    bool alerta = indiceAr < 50;
    Serial.printf("%.1f C  %.0f %%  %.1f hPa  gas %.1f kOhm  indice %.0f\n", t, h, p, gas, indiceAr);
    monitorPublicar(jsonMonitor("BME680 - estacao meteorologica",
      item("Temperatura", t, "C") + "," + item("Umidade", h, "%") + "," + item("Pressao", p, "hPa") + ","
+ item("Gas (VOC)", gas, "kOhm") + "," + item("Indice ar", indiceAr, "%") + "," + item("Altitude", alt,
"m"),
      alerta, "muitos compostos volateis no ar"));
  }
}
```

Experimente

1. Suba um andar de escada com o ESP32 e o celular: a altitude muda ~3 m? A pressão cai ~0,4 hPa?
2. Abra um frasco de álcool a 30 cm: a resistência de gás despenca; quanto tempo leva para recuperar?
3. Cubra o sensor com a mão: umidade e temperatura sobem juntas — e o índice de ar cai (VOC da pele).

Desafios (com alerta por LED RGB e buzzer)

1. **5.1 Barômetro com alerta** — se a pressão cair mais de 3 hPa em 3 h (guarde uma leitura a cada 10 min), LED azul piscando + 2 bipes: “tempestade chegando”.
2. **5.2 Índice de ar** — verde ≥ 75 %, amarelo ≥ 50 %, vermelho < 50 % com buzzer a cada 30 s: “ventile”.

3. **5.3 Altímetro de bolso** — botão zera a altitude de referência; LED muda de cor a cada 5 m subidos; buzzer bipa a cada 10 m.
4. **5.4 Estação completa** — junte BME680 (ar) + DS18B20 (água/solo) + MQ-135 no mesmo ESP32 e na mesma página (I^2C + 1-Wire + ADC convivem).
5. **5.5 Previsão** — implemente a regra de Zambretti simplificada (pressão + tendência → “bom / instável / chuva”) e mostre no celular.

Aula 6 — pH-4502C: medindo a acidez

O que é e para que serve

A **placa pH-4502C** com **sonda de vidro BNC** mede o **pH de 0 a 14** (precisão ~±0,1 após calibração) de água de aquário, piscina, solução hidropônica, solo diluído, sucos, vinagre, leite, sabão… É o instrumento de qualquer laboratório de química, agora ligado ao Arduino.

A química: o que é pH e como um vidro o mede

Na água, uma fração minúscula das moléculas se separa em $\mathbf{H^+}$ e $\mathbf{OH^-}$. $\mathbf{pH = -\log_{10}[H^+]}$: em água pura $[H^+] = 10^{-7}$ mol/L → pH 7. Cada unidade de pH é **10 vezes** mais íons H^+: vinagre (pH 3) tem 10 000 vezes mais H^+ que água. Ácido < 7 < básico (alcalino).

A **sonda** é uma célula eletroquímica: um bulbo de **vidro especial** (silicato com lítio) separa a solução interna (pH 7 fixo) da amostra. Os íons H^+ das duas faces "trocam de lugar" com íons do vidro e geram uma **diferença de potencial** proporcional à diferença de pH — a **equação de Nernst**: E = −59,16 mV × (pH − 7) a 25 °C. Ou seja: **0 mV em pH 7; +177 mV em pH 4; −177 mV em pH 10**. Um segundo eletrodo (de referência, Ag/AgCl no mesmo corpo) fecha o circuito. Esse sinal é minúsculo e de altíssima impedância; a placa 4502C tem um amplificador que o desloca para **0–5 V com pH 7 ≈ 2,5 V** — e é isso que lemos no A0.

Por que calibrar sempre

Cada sonda tem seu **offset** (pH 7 não dá exatamente 2,5 V) e sua **inclinação** (mV por pH cai com a idade e a temperatura). A calibração de **dois pontos** (tampão pH 7 e pH 4 ou 10) resolve ambos: é uma **reta** V = V7 + inclinação × (pH − 7). Repita a calibração toda semana de uso.

Temperatura: a inclinação de Nernst muda 0,2 %/°C (59,16 mV a 25 °C; 54 mV a 0 °C). O pino **To** da placa dá a temperatura — desafio 6.4.

Montagem

Placa pH-4502C	Arduino Uno	ESP32
V+	5V	**VIN** (5 V)
G (os dois)	GND	GND
Po (saída de pH)	A0	1 kΩ → **GPIO 34** → 2 kΩ → GND (divisor)
To (temperatura)	A1 (opcional)	divisor → GPIO 35 (opcional)
Do (limiar)	—	—
BNC	sonda	sonda

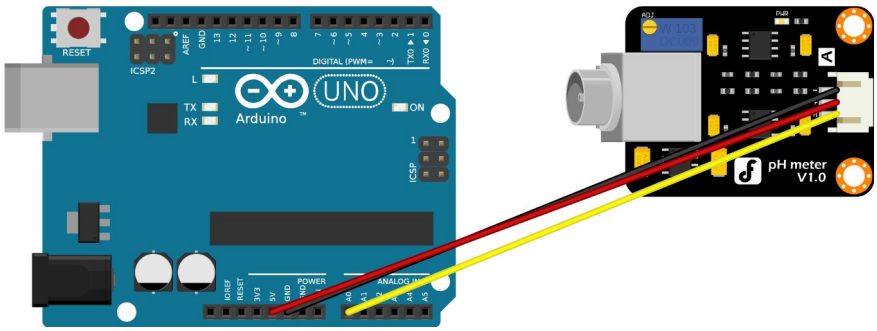


Figura 11 — Aula 6 — pH-4502C no Uno: Po em A0, V+ em 5V. A sonda entra no BNC.

Código (Uno)

06_ph4502c_uno.ino

```
/*
  06 - pH-4502C + sonda BNC: medindo acidez de liquidos
  Placa: Arduino Uno/Nano     Sem biblioteca: e um sinal analogico que NOS calibramos.

  Placa pH-4502C:  V+ -> 5V   G -> GND (os dois G)   Po -> A0 (saida de pH)   To -> A1 (temperatura,
opcional)
                   Do -> saida digital com limiar (pot menor)   A sonda entra no conector BNC.
  Pot grande (perto do BNC) = OFFSET: ajusta a saida para 2,5 V com a sonda em pH 7 (ou com o BNC em
curto).
  Pot pequeno = limiar do Do (nao mexe no Po).

  Quimica: a sonda e uma celula eletroquimica. Uma membrana de VIDRO especial deixa ions H+
  "conversarem" com a solucao; a diferenca de concentracao de H+ entre o liquido e a solucao
  interna (pH 7) gera uma TENSAO (equacao de Nernst): ~59,16 mV por unidade de pH a 25 C,
  e 0 V em pH 7. pH 4 -> +177 mV; pH 10 -> -177 mV. A placa amplifica e desloca esse sinal
  para 0..5 V (pH 7 = 2,5 V). pH = -log10[H+]: cada unidade e 10x mais ions H+.

  Calibracao (2 pontos) com solucoes tampao pH 7,00 e pH 4,00 (ou 10,00):
    1. Sonda em pH 7: digite  c7  no Serial -> grava V7
    2. Enxague com agua destilada, sonda em pH 4: digite  c4  -> grava V4 e calcula a inclinacao
    Os valores ficam na EEPROM. A inclinacao ideal e ~ -0,177 V/pH (depende da placa e da sonda).
*/
#include <EEPROM.h>

const int PINO_PH = A0;
const int N_AMOSTRAS = 30;
float V7 = 2.50;          // tensao em pH 7 (offset)
float INCLINACAO = -0.177;// V por unidade de pH (negativo: mais acido = mais tensao nesta placa)

float lerTensao() {                    // media de N amostras descartando as 5 maiores e 5 menores
  int v[N_AMOSTRAS];
  for (int i = 0; i < N_AMOSTRAS; i++) { v[i] = analogRead(PINO_PH); delay(10); }
  for (int i = 0; i < N_AMOSTRAS - 1; i++) for (int j = 0; j < N_AMOSTRAS - 1 - i; j++)
    if (v[j] > v[j + 1]) { int t = v[j]; v[j] = v[j + 1]; v[j + 1] = t; }
  long soma = 0; for (int i = 5; i < N_AMOSTRAS - 5; i++) soma += v[i];
  return (soma / float(N_AMOSTRAS - 10)) * 5.0 / 1023.0;
}

float tensaoParaPH(float v) { return 7.0 + (v - V7) / INCLINACAO; }

void salvar() { EEPROM.put(0, V7); EEPROM.put(4, INCLINACAO); }
void carregar() {
  float a, b; EEPROM.get(0, a); EEPROM.get(4, b);
  if (!isnan(a) && a > 1 && a < 4) V7 = a;
  if (!isnan(b) && b < -0.05 && b > -0.5) INCLINACAO = b;
}

void setup() {
  Serial.begin(9600);
  carregar();
  Serial.println("Comandos: c7 (sonda em pH 7)   c4 (sonda em pH 4)   c10 (sonda em pH 10)   r (reset
calibracao)");
  Serial.print("V7 = "); Serial.print(V7, 3); Serial.print(" V   inclinacao = "); Serial.print(INCLINACAO,
```

```
4); Serial.println(" V/pH");
}

void loop() {
  if (Serial.available()) {
    String cmd = Serial.readStringUntil('\n'); cmd.trim();
    float v = lerTensao();
    if (cmd == "c7") { V7 = v; salvar(); Serial.print("V7 gravado: "); Serial.println(v, 3); }
    else if (cmd == "c4" || cmd == "c10") {
      float phRef = (cmd == "c4") ? 4.0 : 10.0;
      INCLINACAO = (v - V7) / (phRef - 7.0); salvar();
      Serial.print("inclinacao gravada: "); Serial.print(INCLINACAO, 4); Serial.println(" V/pH");
    }
    else if (cmd == "r") { V7 = 2.5; INCLINACAO = -0.177; salvar(); Serial.println("calibracao padrao
restaurada"); }
  }
  float v = lerTensao();
  Serial.print("tensao = "); Serial.print(v, 3); Serial.print(" V   pH = ");
Serial.println(tensaoParaPH(v), 2);
  delay(1000);
}
```

- `lerTensao()` faz 30 leituras, **ordena, descarta as 5 maiores e 5 menores** e tira a média do resto: é uma **média aparada**, robusta a ruído — o sinal de pH é muito sensível a interferência.
- `tensaoParaPH(v) = 7 + (v − V7) / INCLINACAO`: a reta da calibração. Com a inclinação padrão −0,177 V/pH (o amplificador da placa multiplica os 59 mV por 3).
- Comandos `c7`, `c4`, `c10` gravam os pontos na **EEPROM**: a calibração sobrevive ao desligar.

Calibração passo a passo

1. **Zerar o offset da placa (uma vez):** curto-circuite o BNC (um clipe entre o pino central e a carcaça) e gire o **potenciômetro grande** até `tensao = 2,500 V`. Isso equivale a "pH 7 = 2,5 V". (No ESP32, 1,667 V por causa do divisor.)
2. Prepare as **soluções tampão** (sachês de pH 4,00 / 6,86 ou 7,00 / 9,18 ou 10,01 dissolvidos em 250 mL de água destilada). Elas duram ~1 mês fechadas.
3. Sonda em **pH 7**, espere o valor parar (1–2 min), digite `c7`.
4. Enxágue com água destilada, seque encostando (não esfregue), sonda em **pH 4**, espere, digite `c4`. A inclinação aparece: espere algo entre −0,15 e −0,19 V/pH.
5. Confira com o terceiro tampão (pH 10): erro ≤ 0,1. Se a inclinação estiver longe de −0,177, a sonda está velha ou o BNC está mal encaixado.
6. Meça amostras: vinagre (~2,5–3), suco de limão (~2), refrigerante (~2,5–3), água da torneira (~6,5–7,5), sabão em pó diluído (~10), bicarbonato (~8,3).

Monitor Wi-Fi no celular (ESP32)

O sketch do ESP32 cria a rede **Sensor-pH** (senha `12345678`). Conecte o celular a ela (desligue os dados móveis) e abra **http://192.168.4.1**: a página mostra o valor principal em destaque, os demais valores em cartões, um gráfico das últimas 120 leituras e um aviso de **ALERTA** quando o limite do código é ultrapassado.

- A página é a mesma para todos os sensores (`pagina_web.h`): ela pede `/dados` a cada 2 s e o ESP32 responde um **JSON** com uma lista de itens `{n, v, u}` (nome, valor, unidade).

- O servidor está em `monitor_wifi.h`: `monitorIniciar(nome)`, `monitorAtender()` no `loop()` e `monitorPublicar(json)` quando há leitura nova. Para um sensor novo, só muda o que entra no JSON.
- No ESP32 a calibração é gravada com `Preferences` (a "EEPROM" do ESP32). A página classifica ácido/neutro/básico e alerta fora da faixa 6–8,5 (água de aquário).

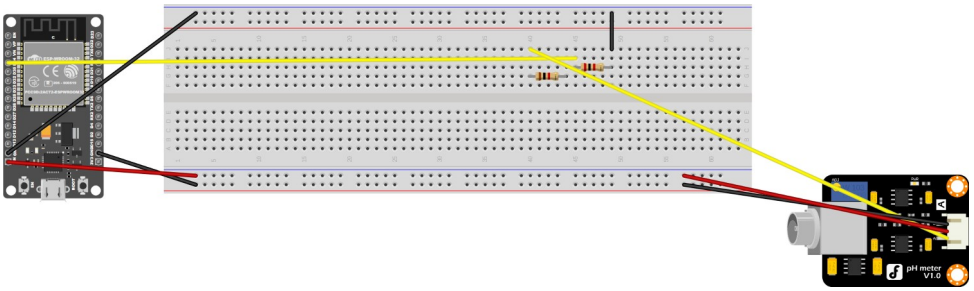


Figura 12 — Aula 6 — pH-4502C no ESP32: V+ no VIN e divisor 1 kΩ/2 kΩ no Po.

06b_ph4502c_esp32_web.ino

```
/*
  06b - pH-4502C no ESP32 com monitor Wi-Fi
  Placa: ESP32 Dev Module     Sem biblioteca (Preferences para guardar a calibracao)
  Arquivos extras: monitor_wifi.h, pagina_web.h

  Ligacoes: V+ -> VIN (5 V; a placa precisa de 5 V)   G -> GND   Po -> 1k -> GPIO 34 -> 2k -> GND
(divisor!)
  A saida Po vai de 0 a 5 V; o divisor entrega 0 a 3,33 V ao ESP32. Calibre com o divisor ja montado
  (a calibracao absorve o fator do divisor e a nao linearidade do ADC do ESP32).
  Comandos pelo Serial (115200): c7, c4, c10, r  - iguais ao sketch do Uno.
  Celular: rede "Sensor-pH" -> http://192.168.4.1
*/
#include <Preferences.h>
#include "monitor_wifi.h"

const int PINO_PH = 34;
const int N_AMOSTRAS = 30;
float V7 = 1.667, INCLINACAO = -0.118;    // valores padrao ja com o divisor (2/3 de 2,5 V e de -0,177)
Preferences prefs;
unsigned long ultimaLeitura = 0;

float lerTensao() {
  int v[N_AMOSTRAS];
  for (int i = 0; i < N_AMOSTRAS; i++) { v[i] = analogRead(PINO_PH); delay(5); }
  for (int i = 0; i < N_AMOSTRAS - 1; i++) for (int j = 0; j < N_AMOSTRAS - 1 - i; j++)
    if (v[j] > v[j + 1]) { int t = v[j]; v[j] = v[j + 1]; v[j + 1] = t; }
  long soma = 0; for (int i = 5; i < N_AMOSTRAS - 5; i++) soma += v[i];
  return (soma / float(N_AMOSTRAS - 10)) * 3.3 / 4095.0;
}
float tensaoParaPH(float v) { return 7.0 + (v - V7) / INCLINACAO; }
void salvar() { prefs.begin("ph", false); prefs.putFloat("v7", V7); prefs.putFloat("incl", INCLINACAO);
prefs.end(); }
void carregar() { prefs.begin("ph", true); V7 = prefs.getFloat("v7", V7); INCLINACAO =
prefs.getFloat("incl", INCLINACAO); prefs.end(); }

void setup() {
  Serial.begin(115200);
  analogReadResolution(12);
  analogSetAttenuation(ADC_11db);
  carregar();
  Serial.printf("V7 = %.3f V   inclinacao = %.4f V/pH\n", V7, INCLINACAO);
  monitorIniciar("Sensor-pH");
}

void loop() {
  monitorAtender();
  if (Serial.available()) {
    String cmd = Serial.readStringUntil('\n'); cmd.trim();
    float v = lerTensao();
```

```
    if (cmd == "c7") { V7 = v; salvar(); Serial.printf("V7 gravado: %.3f\n", v); }
    else if (cmd == "c4" || cmd == "c10") { INCLINACAO = (v - V7) / ((cmd == "c4" ? 4.0 : 10.0) - 7.0);
salvar(); Serial.printf("inclinacao: %.4f\n", INCLINACAO); }
    else if (cmd == "r") { V7 = 1.667; INCLINACAO = -0.118; salvar(); Serial.println("padrao restaurado");
}
  }
  if (millis() - ultimaLeitura >= 1000) {
    ultimaLeitura = millis();
    float v = lerTensao(), ph = tensaoParaPH(v);
    const char* classe = ph < 6.5 ? "acido" : ph > 7.5 ? "basico" : "neutro";
    bool alerta = ph < 6.0 || ph > 8.5;                      // faixa de agua de aquario/piscina, por
exemplo
    Serial.printf("%.3f V  pH %.2f (%s)\n", v, ph, classe);
    monitorPublicar(jsonMonitor("pH-4502C - acidez",
      item("pH", ph, "") + "," + item("Tensao", v, "V") + "," + item("V7 (calib.)", V7, "V"),
      alerta, String("pH fora da faixa: ") + classe));
  }
}
```

Experimente

1. Meça água da torneira e, depois, a mesma água com uma gota de vinagre; depois com uma pitada de bicarbonato. Preveja antes.
2. Deixe a sonda em água com gás: o pH cai devagar ($CO_2 + H_2O \rightarrow$ ácido carbônico).
3. Compare a leitura com fitas de pH (papel indicador): quem é mais preciso?

Desafios (com alerta por LED RGB e buzzer)

1. **6.1 Aquário** — LED verde entre 6,5 e 7,5; amarelo até 6,0/8,0; vermelho + buzzer fora disso, com **histerese de 0,1**.
2. **6.2 Titulação assistida** — buzzer bipa a cada 0,5 de pH que muda enquanto você adiciona bicarbonato a vinagre; LED muda de cor ao cruzar pH 7 (ponto de viragem).
3. **6.3 Calibração pela página** — botões “Calibrar 7” e “Calibrar 4” na página web (rotas `/c7` e `/c4`), com o LED azul piscando enquanto espera estabilizar.
4. **6.4 Compensação de temperatura** — leia o pino To (ou o DS18B20 na amostra) e corrija a inclinação: $incl(T) = incl_{25} \times (273 + T)/298$.
5. **6.5 Registro de titulação no PC** — envie `tempo,pH` pela serial e plote com o script Python da aula de distância; identifique o ponto de viragem no gráfico.

Comparando os seis sensores

Sensor	Grandeza	Precisão	Tempo de resposta	Calibração	Interface	Preço (R$)
DS18B20	temperatura	±0,5 °C	lento (inox): ~30 s	raramente (offset)	1-Wire	10–20
DHT11	temp. + umidade	±2 °C / ±5 %	~10 s; 1 leitura/2 s	software (offset)	1 fio próprio	8–15
MQ-2	gás inflamável / fumaça	indicativa (±30 %)	~10 s; pré-aquec. 3 min–24 h	R0 no ar limpo, sempre	analógica	12–20
MQ-135	qualidade do ar / CO_2 est.	indicativa	idem	R0 no ar externo	analógica	15–25
BME680	T, UR, pressão, VOC	±1 °C, ±3 %, ±1 hPa	8 s (UR); gás ~1 s	fábrica (pressão p_0 do dia)	I²C / SPI	60–120
pH-4502C	pH	±0,1 (calibrado)	1–2 min p/ estabilizar	2 pontos, semanal	analógica	60–100 (c/ sonda)

Três famílias de princípios: **térmicos** (junção de silício, termistor), **capacitivos/mecânicos** (polímero de umidade, membrana de pressão) e **químicos** (óxido metálico aquecido, eletrodo de vidro). Os químicos são os que **mais precisam de calibração** e os que **mais envelhecem** — e são também os que medem o que os outros não conseguem.

Glossário

Termo	Significado
Sensor digital × analógico	Digital entrega um número (DS18B20, DHT, BME); analógico entrega uma tensão que o ADC converte (MQ, pH).
1-Wire / I²C	Barramentos: 1 fio de dados (com endereço de 64 bits) / 2 fios (SDA, SCL) com endereço de 7 bits.
Pull-up	Resistor que mantém um fio em nível alto quando ninguém o puxa para baixo.
Termistor NTC	Resistor cuja resistência cai com a temperatura.
Capacitância	Capacidade de armazenar carga entre duas placas; muda com o material entre elas (água!).
Piezorresistivo	Material cuja resistência muda quando é deformado (membrana de pressão).
Óxido metálico (SnO_2)	Semicondutor cuja resistência cai na presença de gases redutores, quando aquecido.
Rs / R0	Resistência do sensor / resistência no ar limpo; a razão é o que as curvas do datasheet usam.
ppm	Partes por milhão: 1000 ppm = 0,1 % das moléculas do ar.
VOC / IAQ	Compostos orgânicos voláteis / índice de qualidade do ar interno.
hPa	Hectopascal; 1013,25 hPa = pressão padrão ao nível do mar.

Termo	Significado
Ponto de orvalho	Temperatura em que o ar satura e a água condensa.
pH / Nernst	$-\log_{10}[H^+]$ / equação que dá a tensão do eletrodo: 59,16 mV por unidade de pH a 25 °C.
Tampão	Solução de pH conhecido e estável, usada para calibrar.
Offset e inclinação	Os dois parâmetros de uma calibração linear (reta).
Histerese	Limites diferentes para ligar e desligar um alerta, evitando oscilação.
Divisor de tensão	Dois resistores em série que reduzem uma tensão (5 V → 3,3 V com 1 k/2 k).

Anexo — Ficha de calibração

Sensor	Referência usada	Valor de referência	Valor lido	Desvio	Correção aplicada
DS18B20	gelo / fervura	0 °C / 100 °C			
DHT11	sal 75 % / DS18B20				
MQ-2	ar limpo (R0)	Rs/R0 = 9,83	R0 =		
MQ-135	ar externo (R0)	Rs/R0 = 3,6	R0 =		
BME680	INMET / DS18B20				
pH-4502C	tampão 7 / tampão 4	V7 = ; incl. =			

Anexo — Checklist

- [ ] VCC/GND conferidos pela serigrafia (DHT11: módulo ≠ sensor de 4 pinos)
- [ ] Pull-up 4,7 kΩ (DS18B20) / 10 kΩ (DHT11 de 4 pinos)
- [ ] MQ e pH alimentados em 5 V; no ESP32, **divisor** no sinal analógico
- [ ] MQ pré-aquecido antes de calibrar; R0 anotado
- [ ] BME680: endereço 0x76/0x77; SDA/SCL certos (A4/A5 ou 21/22)
- [ ] Sonda de pH úmida, BNC firme, potenciômetro de offset ajustado
- [ ] `monitor_wifi.h` e `pagina_web.h` na pasta do sketch ESP32
- [ ] Celular na rede do sensor com dados móveis desligados